BUKU PANDUAN MODUL PRAKTIKUM KELASIK 2026 PERTEMUAN 3: JAVASCRIPT DASAR, RESPONSIVE DESIGN, DAN TESTING

“Kreasi Digital Tanpa Batas”

Credit by: Sholahuddin Robbani & panitia Kelasik 2026

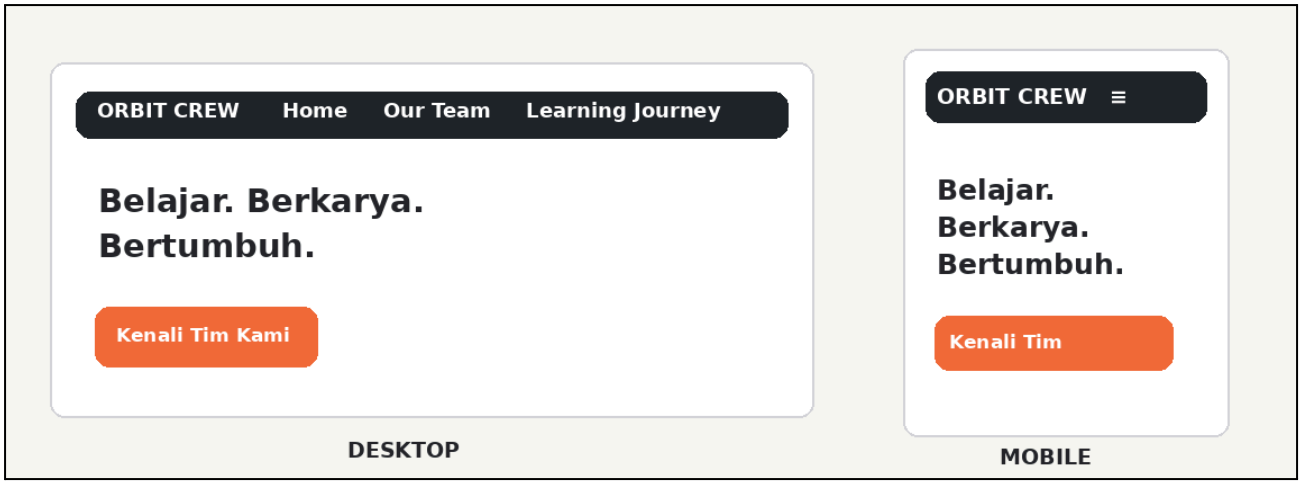


Website yang responsif dan interaktif

MODUL PRAKTIK MENTORING 3 | JavaScript Dasar, Responsive Design, dan Testing

Studi kasus:

Website portfolio kelompok yang dibuat dari desain Figma pada Mentoring 1 dan diimplementasikan dengan HTML/CSS pada Mentoring 2.



Gambar 1 Satu website, susunan berbeda pada layar desktop dan ponsel.

Hasil belajar yang dituju

- Memahami fungsi JavaScript dan cara menghubungkannya ke HTML.
- Membuat menu navigasi mobile yang bisa dibuka dan ditutup.
- Membuat interaksi sederhana untuk konten Learning Journey.
- Mengatur tampilan desktop dan mobile dengan CSS media query.
- Menguji fungsi, tampilan, navigasi, dan aksesibilitas dasar sebelum showcase.

Fokus praktik bersama: menu mobile, satu interaksi JavaScript, dan perbaikan responsive. Penyempurnaan tiga halaman dilakukan kelompok secara mandiri. Tidak menggunakan framework atau backend.

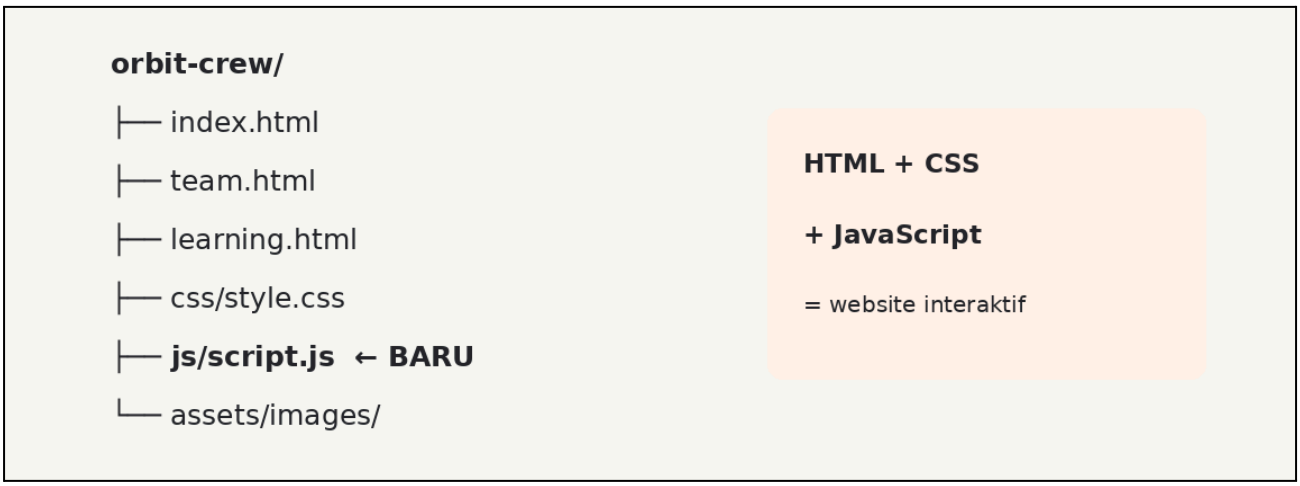
Peta pembelajaran: apa dipelajari dan untuk apa?

Tahap	Apa yang dipelajari	Hasil nyata
1. Persiapan	Cek project HTML/CSS minggu lalu	Tiga halaman dapat dibuka
2. JavaScript dasar	Variabel, DOM, event	Tombol dapat merespons klik
3. Interaksi	Toggle menu & detail materi	Website bisa digunakan
4. Responsive	Viewport, Flexbox, media query	Tampilan tidak rusak di mobile
5. Testing	Checklist, bug fixing	Website siap dipresentasikan

Istilah yang perlu diketahui

- DOM: representasi elemen HTML yang bisa diakses JavaScript.
- Event: kejadian seperti klik tombol atau menekan keyboard.
- Class toggle: menambah atau menghapus class CSS berdasarkan interaksi.
- Responsive: layout menyesuaikan lebar layar, bukan hanya mengecilkan ukuran.
- Testing: mencoba fitur untuk menemukan masalah sebelum digunakan orang lain.

1. Periksa project Mentoring 2



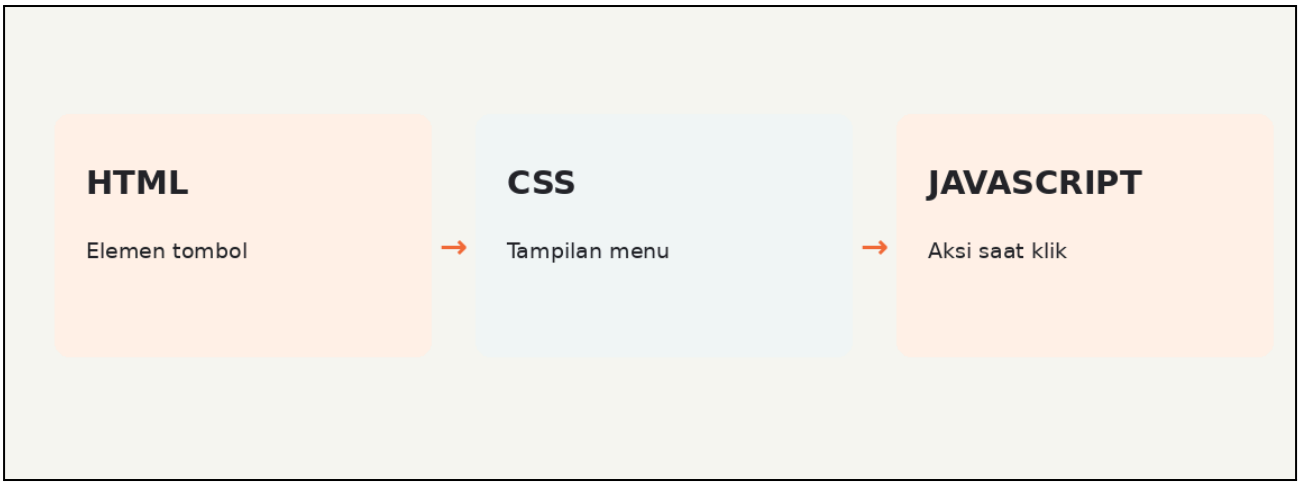
Gambar 1 Tambahkan satu file JavaScript tanpa mengubah struktur halaman yang sudah ada.

1. Buka folder website portfolio di VS Code.
2. Pastikan index.html, team.html, learning.html dan css/style.css masih ada.
3. Coba ketiga halaman di browser; perbaiki link yang rusak terlebih dahulu.
4. Buat folder js dan file script.js.
5. Simpan salinan project atau commit Git sebelum perubahan besar.

CEK HASIL

Folder js/script.js tersedia dan halaman Home, Team, Learning Journey masih bisa dibuka.

2. Mengenal JavaScript tanpa bingung



Gambar 2 HTML menentukan elemen, CSS mengatur tampilan, JavaScript menambahkan perilaku.

Contoh: tombol menu mobile dibuat menggunakan HTML, disembunyikan/ditampilkan dengan CSS, lalu JavaScript menangani aksi saat tombol diklik.

Latihan 01: hubungkan JavaScript ke HTML

Tambahkan baris berikut sebelum tag penutup </body> di ketiga halaman HTML:

```
<script src="js/script.js" defer></script>
```

Isi file js/script.js:

```
console.log("JavaScript berhasil terhubung!");
```

1. Simpan semua file.
2. Buka website di browser.
3. Tekan F12 atau klik kanan → Inspect → Console.
4. Cari tulisan “JavaScript berhasil terhubung!”.

CEK HASIL
Pesan tampil di Console tanpa error berwarna merah.

Konsep dasar yang akan digunakan

```
const namaKelompok = "Orbit Crew";
const jumlahAnggota = 5;
console.log(namaKelompok, jumlahAnggota);
```

const menyimpan nilai yang tidak akan ditugaskan ulang. String adalah teks di dalam tanda kutip; number adalah angka. Pada praktik berikutnya, JavaScript akan memilih elemen HTML dan merespons klik.

3. DOM dan event: membuat tombol bereaksi

JavaScript dapat mencari elemen berdasarkan class atau id. querySelector memilih elemen pertama yang cocok. addEventListener memasang aksi saat suatu event terjadi.

Percobaan kecil sebelum membuat menu

Di dalam <main> halaman Home, tambahkan:

```
<button type="button" id="helloButton">Klik Saya</button>
<p id="helloResult">Belum ada pesan.</p>
```

Di bagian bawah js/script.js, tambahkan:

```
const helloButton = document.querySelector("#helloButton");
const helloResult = document.querySelector("#helloResult");

if (helloButton && helloResult) {
  helloButton.addEventListener("click", () => {
    helloResult.textContent = "Halo! Tombol berhasil diklik.";
  });
}
```

1. Refresh halaman Home.
2. Klik tombol “Klik Saya”.
3. Amati teks berubah tanpa pindah halaman.
4. Hapus percobaan ini setelah paham agar website tetap rapi.

Mengapa ada if? Karena script.js juga dipakai pada halaman lain. Pemeriksaan tersebut mencegah error jika elemen tidak ditemukan.

CEK HASIL

Peserta dapat menjelaskan elemen yang dipilih, event yang dipakai, dan hasil saat tombol diklik.

4. Praktik utama: menu navigasi mobile

Tujuan: di desktop menu tampil biasa; di mobile tersedia tombol “Menu” yang membuka dan menutup tautan navigasi. Gunakan struktur ini secara konsisten pada ketiga halaman.

Langkah A : HTML navbar

Sesuaikan isi <header> pada index.html, team.html, dan learning.html. Pertahankan teks brand kelompok sendiri.

```
<header class="site-header">
  <nav class="navbar" aria-label="Navigasi utama">
    <a class="brand" href="index.html">ORBIT CREW</a>
    <button class="menu-toggle" type="button"
      aria-expanded="false" aria-controls="navLinks">
      Menu
    </button>
    <div class="nav-links" id="navLinks">
      <a href="index.html">Home</a>
      <a href="team.html">Our Team</a>
```

```
      <a href="learning.html">Learning Journey</a>
    </div>
  </nav>
</header>
```

aria-expanded memberi tahu apakah menu terbuka; aria-controls menunjuk elemen yang dikendalikan. Tombol lebih tepat daripada karena bisa digunakan dengan keyboard.

Langkah B: CSS untuk desktop dan mobile

Tambahkan kode berikut di bagian bawah css/style.css:

```
.menu-toggle { display: none; }

@media (max-width: 768px) {
  .navbar { flex-wrap: wrap; }
  .menu-toggle {
    display: inline-block;
    padding: 10px 14px;
    border: 1px solid #fff;
    background: transparent;
    color: white;
    border-radius: 8px;
    cursor: pointer;
  }
  .nav-links {
    display: none;
    width: 100%;
    flex-direction: column;
    gap: 12px;
    padding: 16px 0;
  }
  .nav-links.is-open { display: flex; }
}
```

5. Menu mobile — lanjut: JavaScript dan pengujian

Langkah C: JavaScript untuk membuka dan menutup menu

Tambahkan ke js/script.js:

```
const menuButton = document.querySelector(".menu-toggle");
const navLinks = document.querySelector(".nav-links");

if (menuButton && navLinks) {
  menuButton.addEventListener("click", () => {
    const isOpen = navLinks.classList.toggle("is-open");
    menuButton.setAttribute("aria-expanded", String(isOpen));
  });
}
```

classList.toggle menambah class is-open jika belum ada, dan menghapusnya jika sudah ada. CSS membaca class tersebut untuk menampilkan menu.

Langkah D: Tes menu

1. Buka website pada lebar desktop: tautan menu harus terlihat dan tombol “Menu” tersembunyi.
2. Buka Developer Tools (F12), aktifkan Device Toolbar, pilih ukuran ponsel.
3. Klik “Menu”: tautan Home, Our Team, Learning Journey muncul.
4. Klik “Menu” lagi: tautan tertutup.
5. Klik tautan Team lalu kembali: menu tetap berfungsi di halaman lain.
6. Gunakan tombol Tab dan Enter untuk memastikan tombol bisa dipakai tanpa mouse.

CEK HASIL

Menu berfungsi di tiga halaman; pada mobile tidak ada tautan yang tertutup atau terpotong.

Jika tidak bekerja

- Pastikan file script.js dihubungkan pada setiap halaman.
- Periksa nama class: menu-toggle, nav-links, dan is-open harus sama persis.
- Pastikan media query berada di bawah aturan CSS lama agar tidak tertimpa.
- Buka Console untuk melihat error; jangan menebak tanpa memeriksa.

6. Interaksi kedua: Learning Journey yang bisa dibuka

Fitur ini memberi contoh JavaScript yang relevan dengan isi portfolio: pengunjung dapat melihat atau menyembunyikan catatan tantangan belajar. Gunakan tombol agar akses keyboard tetap baik.

HTML di learning.html

```
<article class="learning-card">
  <h2>Belajar Figma</h2>
  <p>Kami membuat wireframe dan komponen.</p>
  <button class="detail-toggle" type="button"
    aria-expanded="false" aria-controls="figmaDetail">
    Lihat tantangan
  </button>
  <div class="learning-detail" id="figmaDetail" hidden>
    <p>Tantangan: menyamakan spacing tiap halaman.</p>
    <p>Solusi: memakai Auto Layout dan komponen.</p>
  </div>
</article>
```

JavaScript di script.js

```
document.querySelectorAll(".detail-toggle").forEach((button) => {
  button.addEventListener("click", () => {
    const detail = document.getElementById(
      button.getAttribute("aria-controls")
    );
    if (!detail) return;
    detail.hidden = !detail.hidden;
    const open = !detail.hidden;
    button.setAttribute("aria-expanded", String(open));
    button.textContent = open ? "Tutup tantangan" : "Lihat tantangan";
```

```
  });
});
```

CSS opsional

```
.detail-toggle {
  padding: 10px 14px;
  border: 1px solid #f36a38;
  border-radius: 8px;
  background: white;
  cursor: pointer;
}
.learning-detail { margin-top: 16px; }
.learning-detail[hidden] { display: none; }
```

PENTING

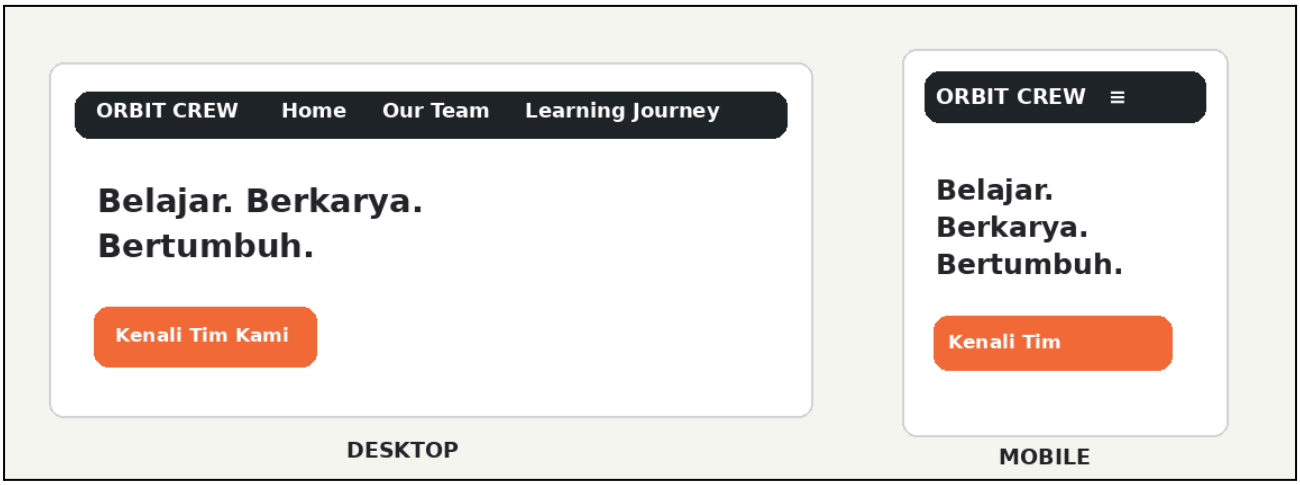
Jika menyalin kartu ini untuk materi lain, ganti id figmaDetail menjadi id unik dan samakan nilai aria-controls. Dua elemen tidak boleh memakai id yang sama.

CEK HASIL

Tombol menampilkan dan menyembunyikan tantangan; teks tombol berubah sesuai keadaan.

7. Responsive design: bukan sekadar mengecilkan halaman

Website responsif menata ulang konten sesuai ruang yang tersedia. Contohnya hero dua kolom pada desktop berubah menjadi satu kolom pada ponsel; kartu anggota dari tiga kolom menjadi satu kolom.



Gambar 3 Perbandingan komposisi layout, bukan sekadar perbedaan ukuran font.

Pastikan meta viewport ada di <head> ketiga halaman

```
<meta name="viewport" content="width=device-width, initial-scale=1.0">
```

Tambahkan CSS responsive

```
img { max-width: 100%; height: auto; }

@media (max-width: 768px) {
```

```
  .hero {
    flex-direction: column;
    padding: 48px 20px;
    gap: 28px;
  }
  .hero h1 { font-size: 34px; }
  .team-grid { grid-template-columns: 1fr; }
  .page-content { padding: 40px 20px; }
}

@media (min-width: 769px) and (max-width: 1024px) {
  .team-grid { grid-template-columns: repeat(2, 1fr); }
}
```

CSS di atas mengasumsikan class .hero, .team-grid, dan .page-content sudah dibuat pada Mentoring 2. Jika nama class berbeda, sesuaikan selector dengan HTML kelompok.

Mengapa perlu breakpoint?

max-width: 768px berarti aturan berlaku saat area tampilan 768 piksel atau lebih sempit. Breakpoint bukan ukuran semua ponsel; pilih berdasarkan kapan layout mulai terasa sempit.

8. Latihan menguji responsive secara mandiri

1. Buka Chrome atau Edge → F12 → aktifkan ikon perangkat (Device Toolbar).
2. Coba lebar sekitar 375px (ponsel), 768px (tablet), dan 1440px (desktop).
3. Pada Home: cek navbar, judul, tombol, gambar, dan jarak hero.
4. Pada Team: cek apakah kartu anggota tersusun rapi dan foto tidak gepeng.
5. Pada Learning Journey: cek apakah tombol detail mudah diklik dan teks terbaca.
6. Geser halaman secara horizontal: seharusnya tidak ada konten yang meluber.
7. Jika bermasalah, identifikasi elemen dan ubah CSS yang menyebabkan masalah.

Perbaikan yang sering diperlukan

- Teks terlalu besar di ponsel → kecilkan font-size heading pada media query.
- Gambar melebar keluar → pakai max-width:100% dan periksa lebar parent.
- Navbar terpotong → gunakan flex-wrap dan menu mobile.
- Kartu terlalu rapat → atur gap dan padding.
- Tombol sulit disentuh → perbesar area klik, bukan hanya teks.

CEK HASIL

Ketiga halaman nyaman dibaca pada ponsel dan desktop, tanpa scroll horizontal yang tidak diperlukan.

9. Testing: periksa fungsi sebelum showcase



Gambar 4 Uji, catat masalah, perbaiki, lalu ulangi pengujian.

Testing bukan sekadar memastikan website tampil. Peserta harus mencoba alur pengunjung dan mencatat jika ada masalah.

Skenario uji pengguna

- Skenario 1: pengunjung membuka Home lalu mencari profil anggota di Our Team.
- Skenario 2: pengunjung membuka Learning Journey lalu melihat tantangan Figma.
- Skenario 3: pengunjung menggunakan website dari ponsel dan membuka menu.
- Skenario 4: pengunjung kembali ke Home menggunakan navigasi.

10. Checklist QA dan tabel pencatatan bug

Lakukan tes bersama anggota kelompok. Tandai setiap poin yang berhasil, lalu tulis masalah yang belum selesai.

- Semua halaman terbuka tanpa error.
- Semua tautan navbar mengarah ke halaman yang benar.
- Menu mobile dapat dibuka dan ditutup.
- Interaksi Learning Journey bekerja.
- Gambar tampil; setiap gambar penting punya teks alt yang sesuai.
- Teks cukup terbaca dan warna memiliki kontras yang layak.
- Navigasi dan tombol bisa diakses menggunakan keyboard.
- Tidak ada scroll horizontal yang tidak perlu pada mobile.
- Console tidak menunjukkan error JavaScript.
- Konten kelompok lengkap dan sesuai rancangan Figma.

Contoh catatan pengujian

Temuan	Penyebab	Perbaikan
Menu tidak muncul di ponsel	Class CSS berbeda	Samakan nav-links dan is-open
Foto anggota terpotong	Ukuran gambar tak konsisten	Gunakan object-fit: cover
Link Team gagal	Nama file salah	Perbaiki href ke team.html
.....	.....	.....

11. Finishing untuk showcase

1. Bandingkan halaman akhir dengan desain Figma: warna, hierarki teks, susunan elemen, dan spacing.
2. Rapikan nama file dan hapus kode percobaan yang tidak digunakan.
3. Pastikan nama anggota, foto, deskripsi, dan Learning Journey benar.
4. Uji website dari perangkat lain atau browser lain jika memungkinkan.
5. Siapkan demo singkat: Home → Team → Learning Journey → menu mobile → fitur interaktif.
6. Jika memakai GitHub, commit perubahan terakhir dan pastikan file terbaru sudah di-push.

Catatan

GitHub Pages dapat digunakan untuk mempublikasikan website statis jika kelompok sudah siap, tetapi publikasi bukan syarat memahami JavaScript dan responsive.

12. Tugas akhir kelompok

Selesaikan website portfolio tiga halaman berdasarkan desain Figma Mentoring 1. Gunakan HTML/CSS dari Mentoring 2, kemudian tambahkan JavaScript, responsive, dan testing dari Mentoring 3.

- Home berisi identitas kelompok, tagline, dan ajakan mengenal tim.
- Our Team menampilkan profil seluruh anggota.
- Learning Journey memuat materi yang dipelajari, penerapan, dan tantangan.
- Navigasi berfungsi di desktop maupun mobile.
- Minimal satu interaksi JavaScript benar-benar bekerja.
- Layout dapat digunakan di ponsel dan desktop.
- Semua aset dan link valid; tidak ada error penting di Console.
- Tersedia bukti pengujian (screenshot desktop/mobile dan daftar temuan).
- Kumpulkan link repository GitHub atau folder project sesuai arahan mentor.

Rubrik penilaian

Kelengkapan konten & navigasi	25 %
Kesesuaian desain Figma & kerapian CSS	25%
Interaksi JavaScript	20%
Responsive design	20%
Testing dan dokumentasi	10%

13. Refleksi dan latihan mandiri

- Apa bedanya HTML, CSS, dan JavaScript dalam website yang kalian buat?
- Mengapa tombol menu menggunakan addEventListener?
- Apa fungsi class is-open?
- Mengapa layout desktop tidak cukup hanya dikecilkan?
- Apa masalah paling penting yang ditemukan saat testing, dan bagaimana cara memperbaikinya?

Referensi belajar

1. MDN JavaScript
   https://developer.mozilla.org/en-US/docs/Learn_web_development/Core/Scripting
2. MDN Responsive Design
   https://developer.mozilla.org/en-US/docs/Learn_web_development/Core/CSS_layout/Responsive_Design
3. MDN DOM Events
   https://developer.mozilla.org/en-US/docs/Learn_web_development/Core/Scripting/Events
4. web.dev Responsive Web Design
   https://web.dev/learn/design/
5. Chrome DevTools Device Mode
   https://developer.chrome.com/docs/devtools/device-mode

PENUTUP

Targetnya bukan membuat website paling rumit, melainkan website yang rapi, mudah digunakan, responsif, dan dapat dijelaskan proses pembuatannya dari Figma sampai browser.